**UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE**

**CARRERA:**

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

**NRC:** 30693

**ASIGNATURA:**

METODOLOGÍA DE DESARROLLO DE SOFTWARE

**TALLER #2**

**INTEGRANTES:**

CAYO JANINE

CHAFLA SANTIAGO

GUAYCHA ANTHONY

MADRID EDMIR

**DOCENTE:**

ING. JENNY RUIZ, MGT.

**12/04/2026**

| | |
|---|---|
| **FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES** | **Departamento:**<br><br>Ciencias de la Computación |
| | **Carrera:** Tecnologías de la Información y la Comunicación<br><br>**Asignatura:** Metodologías de Desarrollo de Software<br><br>**NRC:** 30693 |
| | **Apellidos y nombres del estudiante:**<br><br>Santiago Xavier Chafla Oña |

## ANTECEDENTES

La ingeniería de software surge como disciplina para organizar el desarrollo de sistemas informáticos complejos.

El estudio de los capítulos 1 y 2 permite comprender conceptos básicos como software, metodologías y calidad del producto.

## OBJETIVO GENERAL

Analizar los fundamentos de la ingeniería de software mediante el estudio de los capítulos 1 y 2.

## OBJETIVOS ESPECÍFICOS

- Identificar conceptos clave como software, metodologías y actividades del desarrollo.
- Comprender la importancia de la calidad y los procesos en la ingeniería de software.

Sangolquí, a 12 de abril de 2026

## DESARROLLO

En el presente apartado se abordan los contenidos correspondientes a los capítulos 1 y 2, enfocados en los fundamentos de la ingeniería de software. A través del análisis de conceptos, definiciones y preguntas planteadas, se busca comprender la importancia de los procesos, metodologías y atributos que intervienen en el desarrollo de software, así como su aplicación en la creación de sistemas eficientes y de calidad.

## CAPÍTULO I

### *1.¿Qué es ingeniería de software?*

La ingeniería de software es una disciplina que aplica principios sistemáticos, cuantificables y rigurosos de la ingeniería al desarrollo, operación y mantenimiento del software, con el objetivo de producir sistemas confiables, eficientes y de alta calidad. Se diferencia de la programación tradicional al abarcar todo el ciclo de vida del software, desde el análisis de requisitos y el diseño hasta la implementación, las pruebas, el despliegue y la evolución continúa, utilizando metodologías ágiles o tradicionales para gestionar la complejidad creciente de los proyectos informáticos.

Esta rama integra herramientas, técnicas y prácticas estandarizadas como las definidas por el IEEE para resolver problemas reales en diversos sectores, garantizando no solo funcionalidad, sino también escalabilidad, seguridad y mantenibilidad a largo plazo. En esencia, transforma ideas abstractas en soluciones digitales robustas, minimizando riesgos y optimizando recursos en un mundo cada vez más dependiente de la tecnología. (*¿Qué es la ingeniería de software?*, 2024)

**Referencias:**

*Actividades del la ingeniería de software. Fases o etapas del proceso* . (s/f). Recuperado el 10 de abril de 2026, de https://mwebs.com.uy/blog/actividades-de-la-ingenier%C3%ADa-de-software/31

*Ingeniería de software: Qué es, Objetivos y Funciones*. (s/f). UNIR México. Recuperado el 10 de abril de 2026, de https://mexico.unir.net/noticias/ingenieria/ingenieria-de-software-que-es-objetivos/

***2. ¿Cuáles son las actividades fundamentales de la ingeniería de software?***

Las actividades fundamentales de la ingeniería de software se agrupan en etapas clave del ciclo de vida del desarrollo, presentes en todos los procesos independientemente del tamaño del proyecto. Estas incluyen especificación de requisitos, donde clientes e ingenieros definen las funciones, restricciones y objetivos del software mediante análisis detallado; diseño y desarrollo, que transforma las especificaciones en un sistema ejecutable a través de arquitectura, programación y modelado; y validación y verificación, centrada en pruebas exhaustivas con datos simulados para asegurar calidad y corrección. (*Actividades del la ingeniería de software. Fases o etapas del proceso*, s/f)

Otras actividades esenciales abarcan planificación y gestión, para estimar costos, plazos y riesgos; construcción y despliegue, que integra implementación, integración y puesta en producción; y actividades transversales o "sombrilla" como control de calidad, seguimiento de proyectos, gestión de riesgos y mantenimiento post-entrega. Estas se apoyan en metodologías ágiles o tradicionales, asegurando eficiencia, escalabilidad y adaptabilidad del software en entornos reales. (*Ingeniería de software*, s/f)

**Referencia:**

ISO/IEC/IEEE International Standard - Systems and software engineering–Life cycle management–Part 3: Guidelines for the application of ISO/IEC/IEEE 12207 (software life

cycle processes). (2020). *ISO/IEC/IEEE 24748-3:2020(E)*, 1–76. https://doi.org/10.1109/IEEESTD.2020.9238526

***3. ¿Cuál es la diferencia entre ingeniería de software y ciencias de la computación?***

La ingeniería de software y las ciencias de la computación son disciplinas complementarias pero distintas dentro del ámbito informático. Las ciencias de la computación se centran en la teoría fundamental de la computación, incluyendo algoritmos, estructuras de datos, complejidad computacional, inteligencia artificial y fundamentos matemáticos como la autómatas y la lógica, con un enfoque académico y abstracto orientado a la investigación y el avance del conocimiento. Por su parte, la ingeniería de software aplica principios de ingeniería para el desarrollo práctico y sistemático de software de calidad, enfatizando procesos como el análisis de requisitos, diseño, implementación, pruebas, despliegue y mantenimiento, con énfasis en la escalabilidad, fiabilidad y gestión de proyectos en entornos reales. (Larson, 2021)

**Diferencias Clave**

| **Aspecto** | **Ciencias de la Computación** | **Ingeniería de Software** |
|---|---|---|
| Enfoque principal | Teoría, algoritmos y abstracciones matemáticas | Procesos prácticos y ciclo de vida del software |
| Objetivo | Entender "qué puede computarse" y cómo optimizar soluciones abstractas | Construir software confiable, eficiente y mantenible |
| Habilidades típicas | Matemáticas discretas, investigación y prototipos | Gestión de proyectos, pruebas y metodologías ágiles/waterfall |

| | | |
|---|---|---|
| Aplicación laboral | Investigación, IA, data science | Desarrollo de apps, sistemas empresariales |

En resumen, las ciencias de la computación proporcionan la base teórica que sustenta a la ingeniería de software, la cual se orienta más a la aplicación ingenieril para resolver problemas concretos del mundo real.

**Referencia:**

Larson, E. (2021, mayo 10). *Ciencias computacionales vs Ingeniería en sistemas*. Geek Teacher. https://geekteacher.net/ciencias-computacionales-vs-ingenieria-en-sistemas/

***4. ¿Qué es software?***

El software es el conjunto de programas, instrucciones, datos y reglas informáticas que permiten el funcionamiento de dispositivos electrónicos como computadoras, smartphones o tablets, actuando como el componente intangible opuesto al hardware físico. Compuesto por código ejecutable, bibliotecas y configuraciones, el software proporciona las órdenes precisas que el hardware interpreta para procesar información, ejecutar tareas específicas y facilitar la interacción del usuario con la máquina, desde abrir un documento hasta gestionar redes complejas.

Sin software, el hardware sería inoperable, ya que este último solo responde a señales eléctricas sin inteligencia inherente; ejemplos cotidianos incluyen sistemas operativos como Windows o Linux (software de base), aplicaciones como navegadores web o editores de texto (software de aplicación), y utilidades como antivirus (software de soporte). En esencia, el software transforma dispositivos inertes en herramientas versátiles, evolucionando con actualizaciones para adaptarse a nuevas necesidades y optimizar rendimiento.(*¿Qué Es Software?*, s/f)

**Referencia:**

*¿Qué es la ingeniería de software? Definición, Fundamentos, Características*. (2024, noviembre 21). https://www.guru99.com/es/what-is-software-engineering.html

***5. ¿Cuáles son los atributos del buen software?***

Los atributos de un buen software se refieren a las características de calidad que garantizan su efectividad, eficiencia y utilidad a lo largo de su ciclo de vida, según estándares como ISO/IEC 25010. Estos atributos se dividen en categorías como calidad en uso (usabilidad, eficiencia) y calidad del producto (fiabilidad, mantenibilidad), evaluados mediante pruebas y métricas para asegurar que cumpla expectativas funcionales y no funcionales en entornos reales.(“ISO/IEC/IEEE International Standard - Systems and software engineering–Life cycle management–Part 3”, 2020)

**Atributos Principales**

| **Categoría** | **Atributos Clave** | **Descripción Breve** |
|---|---|---|
| Funcionalidad | Idoneidad, corrección, completitud, pertinencia | Cumple las funciones especificadas con precisión y relevancia para el usuario. |
| Fiabilidad | Madurez, disponibilidad, tolerancia a fallos, recuperabilidad | Opera consistentemente sin fallos bajo condiciones normales y se recupera de errores. |

| | | |
|---|---|---|
| Usabilidad | Reconocimiento de idoneidad, aprendizaje, operabilidad, protección del usuario | Fácil de aprender, usar e interpretar intuitivamente. |
| Eficiencia | Comportamiento temporal, uso de recursos | Optimiza tiempo y recursos (CPU, memoria) en su desempeño. |
| Mantenibilidad | Analizabilidad, modificabilidad, testabilidad, reusabilidad | Permite modificaciones, pruebas y actualizaciones con bajo esfuerzo. |
| Portabilidad | Adaptabilidad, capacidad de instalación, reemplazabilidad | Se transfiere fácilmente entre entornos de hardware/software. |

**Referencia:**

*¿Qué Es Software? ¿Qué Es Un Software As A Service? | Aicad*. (s/f). Aicad Business School. Recuperado el 10 de abril de 2026, de https://www.aicad.es/que-es-software

***6. ¿Cuáles son los principales retos de la ingeniería de software?***

Hacer frente a la creciente diversidad, las exigencias de plazos de entrega reducidos y el desarrollo de software fiable. (Somerville, 2011)

Con la creciente diversidad del software se necesita desarrollar técnicas para poder construir un software confiable. Actualmente muchas técnicas de desarrollo de software consumen tiempo, esto influye el tiempo de entrega para desarrollos de proyectos grandes y

complejos que pueden afectar la calidad del sistema y al mismo tiempo desarrollar un sistema confiable porque el software tiene relación con muchos aspectos de la vida cotidiana.

referencias:

Emery, M. (2020 de agosto de 2020). *Retos en la Ingeniería de Software*. Obtenido de SCRIBD: https://es.scribd.com/document/471826540/Cuales-son-los-retos-fundamentales-que-enfrenta-la-ingenieria-de-software

Sommerville, I. (2011). *Software Engineering.* Boston: Pearson Education.

***7. ¿Cuáles son los costos de la ingeniería de software?***

Aproximadamente el 60% de los costos de software corresponden a costos de desarrollo; el 40% a costos de pruebas. En el caso del software a medida, los costos de evolución suelen superar los de desarrollo.(Somerville, 2011)

***8. ¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?***

Método incremental: al trabajar con ese método se pueden comprobar las mejorar con mayor facilidad, donde cada incremento añade una nueva funcionalidad al sistema hasta que se completa el producto.

Método espiral: combina la naturaleza repetitiva con los aspectos del método cascada, lo que busca es comprobar los avances y poder trabajar con bases sólidas.

Método prototipo: La **metodología de prototipo** parte de la base de lo que entenderíamos como **un borrador** si escribiéramos. El objetivo es realizar un prototipo de **software rápido**, sin reparar en los detalles. (U. Carlemany, 2024)

**Referencias:**

Universitat Carlemany. (03 de marzo de 2024). Metodologías de desarrollo de software: tipos, características y cuán. Obtenido de Universitat Carlemany: https://www.universitatcarlemany.com/actualidad/blog/metodologias-de-desarrollo-de-software

***9. ¿Qué entiende por paradigmas de la ingeniería de software?***

Son agrupación de métodos, herramientas para guiar los procesos que se pueden extender y adaptar para poder crear procesos de ingeniería de software más específicos. Engloban enfoques distintos y metodologías utilizadas en la ingeniería de software para poder enfrentar desafíos en diversos contextos.

**Referencias:**

Sommerville, I. (2011). *Software Engineering.* Boston: Pearson Education.

***10. ¿Cuál es la definición del modelo incremental versus otros paradigmas? Características y diferencias con otras ingenierías.***

**Modelo Incremental:** Se basa en la idea de desarrollar una implementación inicial, exponerla a los comentarios de los usuarios y evolucionarla a través de varias versiones hasta que se haya desarrollado un sistema adecuado (Sommerville, 2005)

**Diferencia con Cascada:** A diferencia del modelo de cascada, el incremental permite mayor flexibilidad ante el cambio de requerimientos y ofrece entregas tempranas, donde permite saltarse los procesos de análisis, diseño, construcción y implementación.

En la Cascada, el cliente no ve nada funcional hasta que el proyecto está casi terminado lo que genera un alto riesgo de rechazo. En el Incremental, el cliente recibe

software funcional rápidamente, lo que reduce el riesgo de que el sistema final no sea lo que el usuario esperaba.

En el Cascada cada fase es crear un documento definitivo y listo para pasar a la otra, en cambio en el incremental no se espera a que cada ciclo tenga un trabajo ya definido, lo que garantiza un software funcional.

**Diferencia con Reutilización:** Mientras el Incremental construye código nuevo por partes, el modelo orientado a la reutilización se enfoca en integrar componentes ya existentes. El modelo Incremental se centra en construir software nuevo de forma progresiva. El modelo de se basa en la integración de componentes existentes o sistemas COTS (comerciales listos para usar).

En el Incremental, se tiene el control total del código para adaptarlo. En el de reutilización, el proceso suele verse limitado por las funcionalidades que los componentes externos ya poseen, obligando a veces a cambiar los requisitos para que encajen con lo que el componente puede hacer.

**Referencias:**

Sommerville, I. (2011). *Software Engineering.* Boston: Pearson Education.

## CAPÍTULO II

**Autoevaluación:**

***1. ¿Qué es una metodología de desarrollo de software?***

- Un conjunto de lenguajes de programación que permiten analizar, diseñar y construir productos software.
- Un conjunto de rutinas de programación que permiten desarrollar aplicaciones de forma ágil.
- Un conjunto de métodos que cubren todo el ciclo de vida de desarrollo de sistemas y que están unidos por un enfoque general filosófico.

**Justificación:** Una metodología de desarrollo de software no se limita solo a programar, sino que abarca todo el proceso: análisis de requisitos, diseño, desarrollo, pruebas y mantenimiento. Además, sigue una estructura organizada y principios definidos que guían cómo se debe trabajar. Por eso, no puede ser solo un conjunto de lenguajes o rutinas, sino un enfoque completo del desarrollo (Valtx, 2022).

**Referencia:**

Valtx. (2022, 19 julio). *Metodologías para el desarrollo de software: ¿Qué son y para qué sirven?* Valtx. https://www.valtx.pe/blog/metodologias-para-el-desarrollo-de-software-que-son-y-para-que-sirven

***2. Las actividades estructurales del proceso de desarrollo de software son:***

Despliegue/comunicación/modelado/diseño.

Gestión de la configuración/modelado.

Administración del proyecto, diseño.

**Justificación:** En la ingeniería de software, las actividades estructurales incluyen procesos clave como la comunicación con el cliente, el modelado del sistema, el diseño de la solución y el despliegue final del software. Estas etapas forman parte del ciclo de vida del software y aseguran que el producto cumpla con los requisitos. Esta opción es la que más se aproxima al modelo propuesto por Pressman (Pressman, 2010).

**Referencia:**

Pressman, R. S. (2010). Ingeniería de software: Un enfoque práctico. McGraw-Hill.

***3. Una de las razones por la cual se incrementan los costos al agregar funcionalidad después que un sistema está en operación es:***

- Lenguaje de programación seleccionado.
- Edad de los programadores.
- Práctica de desarrollo deficiente.
- Fundamentos de Ingeniería de Software un enfoque práctico

**Justificación:** Cuando el software no se diseña correctamente desde el inicio (mala planificación, mala arquitectura o falta de análisis), agregar nuevas funcionalidades después resulta más difícil y costoso. Esto se debe a que se deben modificar partes ya construidas, lo que puede generar errores y retrabajo. Este fenómeno es conocido como el alto costo del cambio en etapas tardías del desarrollo (Turturro, 2023).

**Referencia:**

Turturro, N. (2023, 27 septiembre). *Coste del desarrollo de software a medida: Factores y consideraciones*. DOOR3. https://www.door3.com/es/blog/custom-software-development-cost-factors-and-considerations#:~:text=Alcance%20y%20complejidad:,desarrollo%20de%20software%20a%20medida.

***4. Ingeniería de software asistida por computador significa:***

- Conjunto de herramientas de software integradas para ser utilizadas por otras.
- Conjunto de herramientas de hardware integradas para ser utilizadas por otras.
- Elementos físicos del computador integrados para ser utilizados por otras.

**Justificación:** La ingeniería asistida por computador (CASE) se refiere al uso de herramientas de software que ayudan a automatizar y facilitar actividades del desarrollo, como el diseño, codificación o pruebas. No se trata de hardware, sino de programas que apoyan a otros procesos de software, mejorando la productividad y calidad (Awati & Thibodeau, 2024).

**Referencia:**

Awati, R., & Thibodeau, P. (2024, 23 julio). *What is computer-aided software engineering (CASE)?* Search ERP. https://www-techtarget-com.translate.goog/searcherp/definition/CASE-computer-aided-software-engineering?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

***5. La metodología con sus tipos correspondientes de clasificación es:***

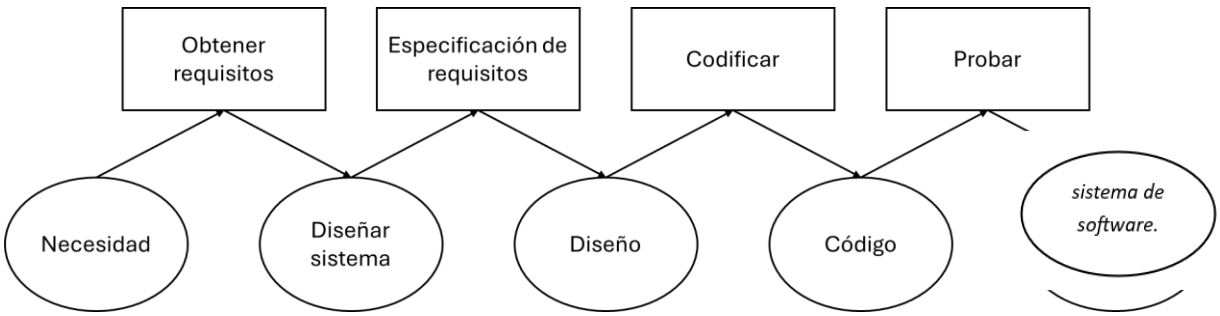
- Metodología Scrum (estructurada).
- Metodología XP (orientada a objetos-ágil).
- Proceso unificado de desarrollo (estructurada).

**Justificación:** XP (Extreme Programming) es una metodología ágil que se basa en prácticas como desarrollo iterativo, programación en pareja y retroalimentación constante. Está fuertemente relacionada con la programación orientada a objetos, ya que promueve código modular, reutilizable y adaptable. Las otras opciones son incorrectas porque clasifican mal las metodologías (De Contenidos de GoDaddy, 2025).

**Referencia:**

De Contenidos de GoDaddy, E. (2025, 26 junio). Metodología XP (Extreme Programming): Qué es y cómo optimizar el desarrollo ágil con prácticas efectivas. GoDaddy Resources - LATAM. https://www.godaddy.com/resources/latam/tecnologia/metodologia-xp-extreme-programming-que-es

***6. En gráfico propuesto, la secuencia ordenada de las figuras es:***




**Justificación:** La secuencia es correcta porque sigue el orden lógico del desarrollo de software: inicia con la necesidad, continúa con la obtención y especificación de requisitos, luego el diseño, la codificación y las pruebas, hasta obtener finalmente un sistema de software funcional que cumple con lo planteado.

**Referencia:**

https://mwebs.com.uy/blog/actividades-de-la-ingeniería-de-software/31

***7. Los modelos de calidad del producto software y los modelos de calidad del proceso de desarrollo de Software son:***

- ISO/IEC 25010 producto software.
- ISO/IEC 20000 producto software.
- ISO 9126 proceso de desarrollo.

**Justificación:** El estándar ISO/IEC 25010 pertenece a la familia **SQuaRE (Software Quality Requirements and Evaluation)** y está específicamente diseñado para evaluar la **calidad del producto software**. Define un modelo con características como:

- Funcionalidad

- Usabilidad
- Fiabilidad
- Eficiencia
- Seguridad
- Mantenibilidad
- Portabilidad

Estas características permiten medir directamente **qué tan bueno es el software como producto final**.

En cambio:

**ISO/IEC 20000** → Está orientado a la **gestión de servicios de TI**, no a la calidad del producto software.

**ISO 9126** → Es un modelo anterior que también trataba la calidad del producto, pero fue **reemplazado por ISO/IEC 25010**, por lo que ya no es el estándar vigente principal.

**Referencia:**

ISO/IEC 25010:2011. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*.

**8.** La norma ISO/IEC 25010 identifica características y subccaracterísticas de calidad del producto software.

La opción correcta es:

- Seguridad es subcaracterística.
- Eficiencia de desempeño es característica.
- Fiabilidad es subcaracterística.
- Fundamentos de Ingeniería de Software un enfoque práctico

**Justificación:** La selección de la opción “Eficiencia de desempeño es característica” se fundamenta en lo establecido por la norma ISO/IEC 25010, la cual define el modelo de calidad del producto software estructurado en características y subccaracterísticas.

En dicho modelo, la eficiencia de desempeño se clasifica como una característica principal, es decir, un atributo de alto nivel que permite evaluar el comportamiento del sistema en relación con el uso de recursos y el tiempo de respuesta. Esta clasificación la diferencia de las subccaracterísticas, que corresponden a desgloses más específicos dentro de cada característica.

**Referencia:**

Pressman, R. S. (2010). *Ingeniería de software: Un enfoque práctico*. McGraw-Hill.

**9.** Según los roles y las responsabilidades correspondientes, qué opción es correcta.

Cambiar el código según las actividades de desarrollo estándar o las solicitudes de cambio corresponde a un auditor.

Mantener la configuración del código corresponde a un configuration manager.

Verificar los cambios y resolver conflictos corresponde a un desarrollador.

**Justificación:** En un equipo de desarrollo de software moderno (especialmente en metodologías ágiles y DevOps), los **roles** tienen responsabilidades muy claras y diferenciadas:

- Desarrollador (Developer): Es quien escribe, modifica, verifica y prueba el código. Sus tareas incluyen:
- Implementar cambios según requerimientos o actividades de desarrollo estándar.
- **Verificar los cambios** que él mismo realiza (revisión de código, pruebas unitarias, etc.).

- **Resolver conflictos** de merge (merge conflicts) cuando integra su código con el del resto del equipo (esto es una tarea típica y cotidiana del desarrollador en Git u otros sistemas de control de versiones).
- Hacer pull requests, revisar su propio código o el de pares, y asegurar que el código funcione antes de integrarlo.

➢ **Configuration Manager** (o Release Manager / DevOps Engineer en algunos contextos): Se encarga de **mantener la configuración** del código, entornos, pipelines de CI/CD, versiones, branches, etc. No suele ser quien resuelve conflictos de código a nivel diario.

➢ **Auditor**: Su rol es de **supervisión y verificación independiente**. Revisa que los procesos se sigan correctamente, que haya trazabilidad de cambios, cumplimiento de estándares, etc. **No** es quien cambia código ni resuelve conflictos técnicos.

**Referencia:**

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

***10. Según los objetivos con la correspondiente fase del proceso unificado de desarrollo, qué opción es la correcta.***

- Inicio establece el alcance del proyecto y las condiciones de los límites, criterios de aceptación, lo que se necesita en el producto y lo que no.
- Transición completa el análisis, diseño, desarrollo y prueba.
- Construcción logra versiones de código útiles tan rápido como se pueda.

**Justificación:** La fase de Inicio se encarga de definir qué se va a hacer (alcance, límites y criterios de aceptación), mientras que las fases posteriores se encargan de cómo se va a construir y entregar el producto. Esta es la base fundamental del Proceso Unificado.

**Referencia:**

Kruchten, P. (2004). *The Rational Unified Process: An Introduction*. Addison-Wesley.

## CONCLUSIONES

1.-La ingeniería de software es una disciplina fundamental en el desarrollo de sistemas informáticos, ya que permite aplicar métodos estructurados y organizados que garantizan la calidad, eficiencia y fiabilidad del software a lo largo de todo su ciclo de vida.

2.-El estudio de los capítulos analizados permitió comprender la importancia de conceptos clave como las metodologías de desarrollo, los atributos del buen software y las actividades fundamentales, los cuales contribuyen a la creación de productos tecnológicos más seguros, escalables y mantenibles.

## RECOMENDACIONES

1.-Se recomienda aplicar metodologías de desarrollo adecuadas según el tipo de proyecto, con el fin de mejorar la organización del trabajo, optimizar recursos y asegurar el cumplimiento de los requisitos establecidos.

2.-Es importante considerar los estándares de calidad desde las primeras etapas del desarrollo de software, ya que esto permite reducir errores, facilitar el mantenimiento y garantizar un mejor desempeño del sistema en entornos reales.

## ANEXOS

**Tabla de integrantes del Taller 2:**

| |
|---|
| Anthony Sebastian Guaycha Ñauñay |
| Ashley Janine Cayo Molina |

Edmir Geovanny Madrid Bermeo